Phuthiphong Wongchantib 660610840

CounterClockwise-Only Steering Robot — Detour Steer Algorithm

Drive on, steer less, come back

Robot. One DC motor: turning it one way *steers* the wheel module, *counter-clockwise only* (one-way clutch); turning it the other way *drives* forward. It can never steer and drive at the same time. The old method steers straight at the goal, so a goal a little to the *right* (φ close to 360°) costs almost a full turn. **Idea.** Two straight legs whose headings differ by more than 180° can reach *any* direction, so the robot may instead drive a straight on, steer only β^* ($180^\circ < \beta^* < \varphi$), and drive b to the goal. The best β^* has a closed form. Headings are counter-clockwise from the $+x$ axis. ω steer speed, v drive speed, t_s settle time, t_b stop time (drive $\rightarrow$ steer), *tol* tolerance, r_{goal} arrival radius. **On the robot** (`mor_luam`, ESP32) the module turns counter-clockwise seen from above: the same frame as this page. The firmware helper name cwErrorDeg is historical; its sign is counter-clockwise. Algorithms 1.1–1.5 plan; 1.6–1.7 are how the robot steers and drives each leg.

Part 1. The algorithm: why it is faster (proof and ideal simulation)

Algorithm 1.1: Counter-Clockwise Heading Helpers

```
Procedure : Bearing(Δx, Δy)                    // CounterClockwise heading that points along (Δx, Δy)
    return atan2(Δy, Δx) mod 2π;
end
Procedure : SteeringError(θ, h)                // How far the wheel must turn counter-clockwise: 0 ... 2π
    return (θ − h) mod 2π;
end
Procedure : InTolerance(e)                     // Close enough on either side of the target
    return e ≤ tol or e ≥ 2π − tol;             // Just past the target reads as e ≈ 2π
end
```

Algorithm 1.2: Optimal Detour Angle in Closed Form, $O(1)$

```
Procedure : OptimalDetourAngle(φ, d)           // φ: counter-clockwise angle to the goal, d: distance
    if φ ≤ π then return None;                 // Theorem 1: steering straight is optimal
    k ← (d ω / v) |sin φ|;                     // One number decides everything
    if k ≥ 2 then return None;                 // Theorem 3: time keeps falling up to β = φ
    β* ← 2π − arccos(k − 1);                   // The only root of dT/dβ = 0 in (π, 2π)
    if β* ≥ φ then return None;                // The best angle is not a detour
    return β*;
end
```

Algorithm 1.3: Plan One Leg: Direct or Detour, $O(1)$

Procedure : *PlanLeg(φ, d)* // Returns (kind, a, β, b, time)
  $T_D \leftarrow \varphi/\omega + t_s + d/v$; // Old method: steer φ, drive d
  $\beta^* \leftarrow$ *OptimalDetourAngle(φ, d)*;
  if $\beta^* =$ **None then return** (DIRECT, 0, φ, d, T_D);
  $a \leftarrow d\sin(\beta^* - \varphi)/\sin\beta^*$; // Leg 1: heading unchanged, no steer
  $b \leftarrow d\sin\varphi/\sin\beta^*$; // Leg 2: heading β^*, ends at the goal
  $T_R \leftarrow a/v + t_b + \beta^*/\omega + t_s + b/v$; // One extra stop between the legs
  if $T_R < T_D$ **then return** (DETOUR, a, β^*, b, T_R); // Use the detour only if it is really faster
  return (DIRECT, 0, φ, d, T_D);
end

Algorithm 1.4: Closed-Loop Detour Steer to One Goal (plan again after every move)

Procedure : *DetourSteer(p, h, g)* // p: position, h: wheel heading, g: goal
  while $\|g - p\| > r_{goal}$ **do**
    $d \leftarrow \|g - p\|$; $e \leftarrow$ *SteeringError(Bearing($g - p$), h)*;
    if *InTolerance(e)* **then** // Already aimed: drive to the goal
      DriveReaim(d); // Algorithm 1.7
    else
      plan $\leftarrow$ *PlanLeg(e, d)*;
      if *plan.kind* $=$ DETOUR **then**
        DriveReaim(plan.a); // Leg 1: straight on, no steer
        Stop(); // t_b: switch the motor from drive to steer
      else
        SteerOneWay(Bearing($g - p$)); // Algorithm 1.6: stops a little short on purpose
      end
    end
    $p \leftarrow$ *MeasurePosition()*; $h \leftarrow$ *MeasureHeading()*; // AS5600 wheel angle + IMU body heading; T4
    // Short by ε: $e = \varepsilon$, a small extra steer. Past by ε: $e = 2\pi - \varepsilon$, PlanLeg detours
  end
  return (p, h);
end

Algorithm 1.5: End-to-End Route Following with Detour Steer

Procedure : *FollowRouteDetour(W)* // Master procedure: visit the goals W in order
  $p \leftarrow (0, 0)$; $h \leftarrow 0$; // Start pose
  foreach $g \in W$ **do**
    $(p, h) \leftarrow$ *DetourSteer(p, h, g)*;
  end
  Display total time; // Goal 0.30 m at 354°: a=0.31 m, β^*=249°, b=0.034 m, 18.83 s (old method: 20.16 s)
end

Algorithm 1.6: Steer One Way: Stop a Little Short on Purpose (robot, every 10 ms)

Procedure : *SteerOneWay*(θ) // θ: the heading to reach
  while true **do** // Every control tick, 10 ms
    $h \leftarrow$ *MeasureHeading()*; // Wheel (AS5600) + body (IMU): the body twists as it steers
    $e \leftarrow$ *SteeringError*(θ, h); $r \leftarrow$ steering speed over 50 ms;
    if $e < \pi$ **and** $e \leq e_{land} + r\,c$ **then break**; // Cut early: it coasts on to $\approx e_{land}$ short
    if *InTolerance(e)* **and** ($e > \pi$ **or** $r < r_{min}$) **then break**; // Past, or standing: a push from rest jumps past
    steering motor on;
  end
  steering motor off; wait until the wheel stops and the body turns $< 5^\circ$/s (at most 0.4 s);
  $c \leftarrow (1-\lambda)\,c + \lambda$ (angle coasted$/r_{cut}$); // Learn the coast time c
end

Algorithm 1.7: Drive with Early Re-aim (one motor: it cannot steer while it drives)

Procedure : *DriveReaim*(ℓ) // ℓ: leg length on the current heading h
  $q \leftarrow p + \ell\,(\cos h, \sin h)$; // The end of this leg
  while true **do** // Every control tick, 10 ms
    $(f, s) \leftarrow q - p$ seen from the wheel; // f ahead, s beside (+: steering side)
    if $\|q - p\| \leq r_{goal}$ **or** $f \leq 0$ **then break**; // Arrived, or level with it
    if $s > r_{goal}$ **and** $f \leq \max(m, R\,s)$ **then break**; // Re-aim is then only $\arctan(1/R)$
    drive motor on;
  end
  drive motor off; $p \leftarrow$ *MeasurePosition()*; // Algorithm 1.4 plans again from here
end

Proof that Detour Steer is never slower, and how much faster it is (ideal model)

For a goal at counter-clockwise angle $\varphi > \pi$ and distance d, a detour with final heading β takes

$$T(\beta) = \frac{\beta}{\omega} + t_s + t_b + \frac{d}{v}\, g(\beta), \qquad g(\beta) = \frac{\sin(\beta - \varphi) + \sin\varphi}{\sin\beta}, \qquad g'(\beta) = \frac{\sin\varphi}{1 + \cos\beta}.$$

T1. If $\varphi \le 180°$, steering straight at the goal is optimal. Two headings less than 180° apart only reach directions between them, so every path steers at least φ; every path also drives at least d. The direct path does both.

T2. If $\varphi > 180°$, a best path has at most 2 legs, with headings 0 and β. The shortest way to write the goal vector with headings in $[0, \beta]$ uses the two headings on either side of φ (the optimum of a 2-row linear program has 2 non-zero variables).

T3. On $(\pi, 2\pi)$, $T'(\beta) = 1/\omega - (d/v)|\sin\varphi|/(1 + \cos\beta)$ is strictly increasing, so T has exactly one minimum: $\cos\beta^* = k - 1$ with $k = (d\omega/v)|\sin\varphi|$. If $k \ge 2$, T keeps falling up to $\beta = \varphi$, which is the direct path.

T4. After leg 1, *PlanLeg(β^*, b)* returns "steer β^*, drive b". Any other detour from there is a longer leg 1 plus one more stop t_b, which T3 already beat. So the planner can run again after every move (closed loop) without changing its mind.

O. *PlanLeg()* is $O(1)$ in time and memory: 0.52 µs per call in Python, against 505 µs for a 3600-point search over β (973×). T1–T4 are also checked by brute force in `detour_steer.py`: closed form vs a 20 000-point grid, vs every 2-leg path on a 2° grid, and 2000 replans. All PASS.

Unknown stop error. Nobody knows in advance whether the wheel stops a little past or a little short. The robot reads the real wheel angle (AS5600) and body heading (IMU) after it stops, and *DetourSteer()* plans again from there; *PlanLeg()* never uses the error. Inside $\pm tol$ the robot drives on its real heading, and a miss of more than r_{goal} is planned again.

Why stop short on purpose (Algorithm 1.6). Past its angle by more than *tol*, a one-way wheel needs almost a full turn; short of it, only a little more. So the robot cuts the motor early and lets the wheel coast to $e_{land} = 2°$ short of the angle (c, the coast time, is learned on every steer). A wheel standing inside the band is never pushed again: from rest the motor must build up power and the wheel jumps past (measured: 9°). Stopping short also means a miss always leaves the goal on the *steering side* of the line the robot drives on, so *DriveReaim()* fixes it with a small turn ($\arctan(1/R) = 18°$ for $R = 3$, $m = 2$ cm) instead of a 90° turn after passing the goal.

Results in simulation, at the robot's own speeds ($v = 0.03$ m/s, $\omega = 35°/$s, $t_b = 0.2$ s, $r_{goal} = 2.5$ mm, the values measured on `mor_luam`; 5000 random goals, $d \in [0.3, 2]$ m; the same goals and stop errors for both; mean time per goal):

wheel stops	bias	sd	old method (s)	detour (s)	saved
exactly	0°	0°	43.40	43.40	0.0 %
either way, random	0°	1°	48.33	48.10	0.5 %
either way, random	0°	2°	50.10	49.60	1.0 %
either way, random	0°	3°	53.00	51.75	2.4 %
mostly short	−2°	2°	48.21	48.08	0.3 %
mostly past	+2°	2°	57.55	55.09	4.3 %
mostly past	+2°	3°	60.44	56.88	5.9 %

At the robot's speeds Detour Steer was slower on 0 % of the goals in every case, but the gain on random goals is small (0–5.9 %). The robot drives slowly, so driving takes most of the time, and a detour is chosen only for goals just to the right: from $\varphi \ge 346°$ at $d = 0.30$ m and $\varphi \ge 356°$ at $d = 1$ m. For the goal of the robot test (0.30 m at 354°) the model gives 18.83 s against 20.16 s (6.6 %, Figure 1a).

Other speeds (the same 5000 goals and stop errors, sd 2°; time saved by Detour Steer, Figure 2):

drive \ steer	20°/s	35°/s	60°/s	120°/s
0.03 m/s (the robot)	2.3 %	1.0 %	0.4 %	0.1 %
0.10 m/s	5.4 %	3.8 %	2.2 %	0.6 %
0.25 m/s	6.7 %	5.7 %	4.2 %	2.2 %

A faster drive or a slower steer makes the detour pay more. At the robot's drive speed with steering at 20°/s, 0.2 % of the goals were a little slower with Detour Steer (a stop error after the extra stop); at 35°/s and faster, none were.

The 2 m route (Figure 3). A total of 2 m split into 7 equal legs (2/7 m each), each leg 6° to the right of the one before; both methods visit the same points. At the robot's speeds the simulation gives:

	time (s)	path (m)	steered	detours
old method	137.82	2.00	2478°	0
Detour Steer	110.52	2.17	1291°	4
saved	27.30 (19.8 %)			

Every leg is 6° to the right, so the old method steers about 354° at each point; Detour Steer drives on and steers less at 4 of the 7 points.

Closest prior work, and the difference. One-motor steer-and-drive mechanisms (US 8,424,627 B2, 2013; Sekine & Mizuuchi, ICINCO 2022) give no planner. Dubins cars that can turn one way only (Bakolas & Tsiotras, JOTA 2011; Patsko & Fedotov, 2018–2019) turn *while moving*. Turn-cost routing (angular-metric and quadratic TSP) orders waypoints, and predictive coast stopping (US 8,571,394 B2) stops early. None of the work found gives this closed-form detour for a robot that must *stop to steer, counter-clockwise only*. "Not found" is not proof that nobody did it.

Part 1 conclusion. Detour Steer is proven never slower than steering straight (T1–T4) and plans in $O(1)$. In the ideal model at the robot's speeds it is 6.6 % faster on the robot-test goal, saves 0–5.9 % on random goals (it detours only for goals just to the right), and 19.8 % on the 2 m / 7-leg route, where every point is a little to the right.

Figure 1. Simulation at the robot's speeds: two goals 0.30 m away

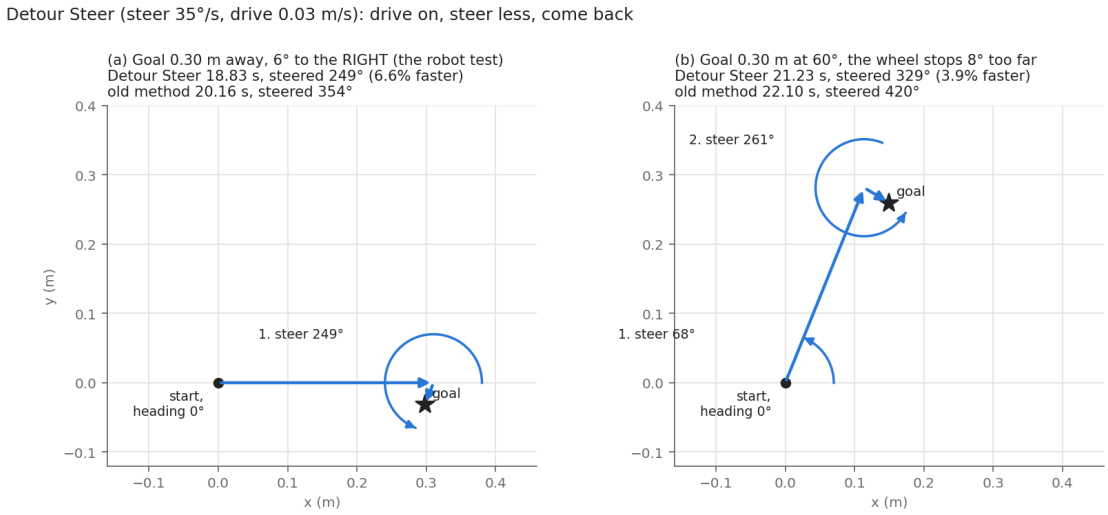
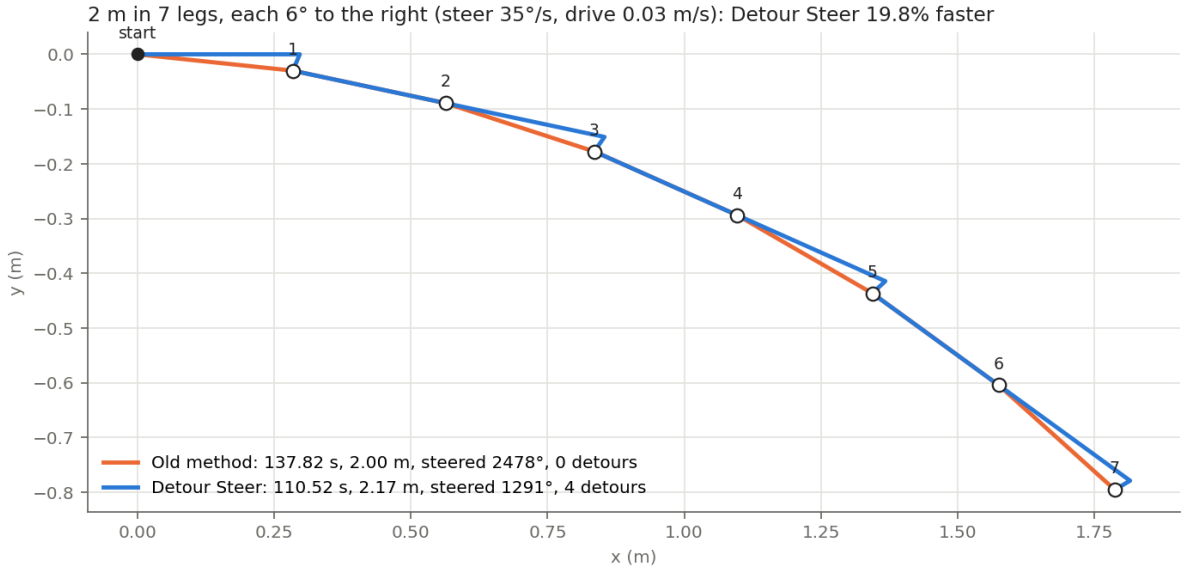


Figure 2. Other speeds: time saved on the same 5000 goals



Figure 3. Simulation of the 2 m / 7-leg route at the robot's speeds



Part 2. The real robot: measured results and robot-like simulation

Results on the robot (`mor_luam`: $v = 0.03\,\mathrm{m/s}$, $\omega = 35^\circ/\mathrm{s}$ its measured steering speed, $t_b = 0.2\,\mathrm{s}$, $r_{goal} = 2.5\,\mathrm{mm}$). The same task for both: before the clock starts the wheel is turned to the start heading and is still; the goal is then placed $d = 0.30\,\mathrm{m}$ away at $\varphi = 354^\circ$ from where the wheel really points. The robot times each run itself and drives back to the start (not timed). 3 rounds, order old–detour, detour–old, old–detour:

round	old method (s)	Detour Steer (s)	saved (s)
1	24.17	20.83	3.34
2	22.94	20.77	2.17
3	23.36	22.33	1.03
mean	23.49	21.31	2.18 (9.3 %)

Detour Steer was faster in 3 of 3 rounds. It steered 3.8 s less (8.5 vs 12.4 s) and drove 1.2 s more (11.8 vs 10.6 s). The plan predicted 18.8 vs 20.2 s; the floor gave a bigger gap, because the old method's near-full turn also needs more correction at the end. All 6 runs ended within 2.5 mm of the goal.

Why the robot drifts right (both methods). (1) One motor steers and drives, so the robot cannot correct its course while it drives: any drift adds up until it stops and re-aims (Algorithm 1.7). (2) Steering and driving act on each other through the same gear train: the drive torque turns the wheel module and its gears a little (while it drives, the AS5600 wheel angle keeps changing and the front of the robot turns), and a near-full steer twists the body about 4°. (3) The wheel slips, the floor is not flat, and the robot's weight is not balanced. (4) The robot stops steering about 2° short on purpose (past would cost almost a full turn); it steers counter-clockwise, so short means the wheel points a little to the right of the goal. Together the body drifts about 3° while driving (IMU), and both methods end 1–3 cm right of their line; the early re-aim (about 18°) brings them back.

How far the simulation is from the robot. The only route run on both is the robot test (0.30 m at 354°, 3 rounds each). The simulation uses the robot's measured speeds ($v = 0.03\,\mathrm{m/s}$, $\omega = 35^\circ/\mathrm{s}$, $t_b = 0.2\,\mathrm{s}$, $r_{goal} = 2.5\,\mathrm{mm}$):

	simulation (s)	robot, mean of 3 (s)	error
old method	20.16	23.49	−3.33 s (−14.2 %)
Detour Steer	18.83	21.31	−2.48 s (−11.6 %)
time saved	1.33 (6.6 %)	2.18 (9.3 %)	

The simulation is 11.6–14.2 % faster than the robot, because it has no drift, no slip and no re-aim stops (see "Why the robot drifts right"). It also under-states the gain: the robot saved 9.3 %, the simulation 6.6 %, because the old method's near-full turn needs more correction on the floor.

Robot-like simulation from measured values. The robot's own logs (every 10 ms; 15 big turns, 5 small turns and 23 drives, `tools/test_out`) show how it really moves. Each turn and drive in the simulation is drawn at random from these measured spreads, so no two runs are the same (1000 runs each; range = 5–95 %):

- steering time (motor on) = 0.90 s + angle/37.8°/s, scatter sd 0.19 s (turns of 3–4°: 0.5–0.65 s); the speed is not constant: it ramps up, runs at 30–50°/s and slows down at the end;
- after a steer the robot waits 0.11 s (measured), plus up to 0.4 s for the IMU in the current firmware;
- the wheel stops 2° short on purpose, spread 0.15° (measured);
- while driving, the direction of travel turns right $14.4 \pm 14.5^\circ/\mathrm{m}$: the body $7.4 \pm 7.1^\circ/\mathrm{m}$ (IMU) plus the wheel module $7.0 \pm 12.6^\circ/\mathrm{m}$ (AS5600), as the drive torque turns the module and gears;
- early re-aim (Algorithm 1.7);
- overshoot: a turn that ends past its angle needs almost another full turn. None of the 20 logged turns overshot, but it does happen on the robot, so it is tested with a probability p per turn.

	old method (s)	Detour Steer (s)
robot, 3 rounds	23.49 (22.94–24.17)	21.31 (20.77–22.33)
model, $p = 0$	23.95 (22.09–26.11), +2.0 %	18.42 (16.83–20.99), −13.6 %
model, $p = 3\,\%$	24.67 (22.15–33.94), +5.0 %	18.67 (16.73–21.93), −12.4 %
model, $p = 8\,\%$	26.11 (22.22–36.41), +11.2 %	18.99 (16.73–25.79), −10.9 %

With measured values only (nothing fitted) the old method matches the robot within 2 %. Detour Steer on the robot is about 2.9 s slower than the model: every real Detour run ends with one more small turn and a short drive (Figure 4, the last pink and blue), which the model does not produce. The logs hold no Detour run, so the cause of this end correction cannot be measured yet; a logged Detour run would close the gap. This model

therefore over-states the gain (23 % against 9.3 %) and is not used to claim a number for the robot. Overshoot makes the old method slower (each one costs almost a full turn), while Detour Steer turns such a miss into a detour. The simulator window has the same mode ("robot-like" check box): each press of Start is one new random run, and it shows the mean and range of 1000 runs.

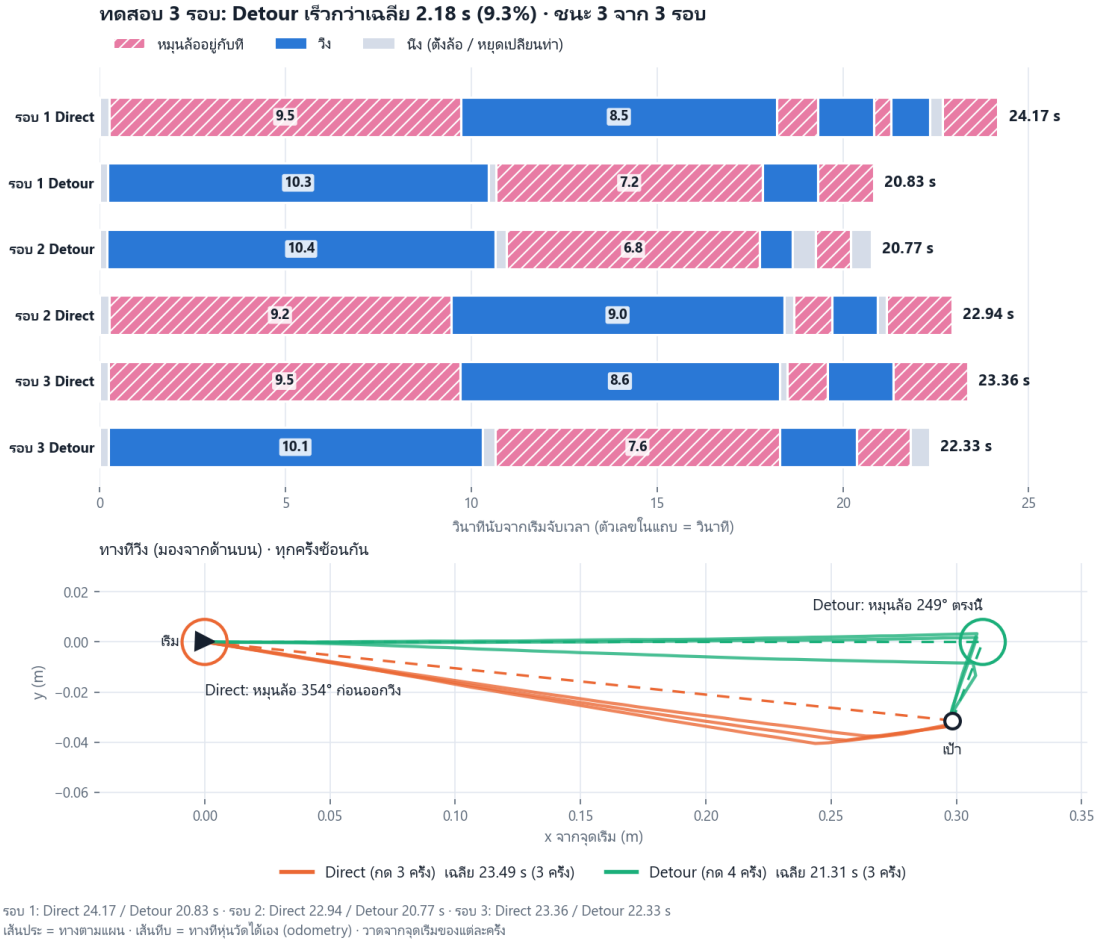
The 2 m route in the robot-like model. This route was run in simulation only, not on the robot. In the measured model ($p = 0$, 1000 runs each) the gain depends on how close the robot must get to each point (`navTolM`); the robot test used 2.5 mm:

point tolerance	old method (s)	Detour Steer (s)	saved
2.5 mm (as tested)	154.6 (150.8–158.1)	143.1 (132.1–148.2)	7.4 %
1 cm	150.5 (147.4–153.4)	130.4 (105.5–142.7)	13.4 %
2 cm	149.7 (146.8–152.4)	118.1 (80.7–139.9)	21.1 %
6 cm	144.5 (133.4–148.7)	108.0 (76.4–135.8)	25.3 %

With a 2.5 mm tolerance both methods spend much of their time on small corrections at each of the 7 points. Because the model is too fast for Detour Steer on the 0.30 m test, these gains are upper values. The return to the start (about 1.96 m) is not timed.

Part 2 conclusion. On the real robot Detour Steer was faster in 3 of 3 rounds: 21.31 s against 23.49 s (9.3 %); this measured result is the one to trust. The robot is 11.6–14.2 % slower than the ideal model because of drift, the turning wheel module, slip and re-aim stops. A simulation built only from the robot's logs matches the old method within 2 % but is about 2.9 s too fast for Detour Steer (one end correction it does not reproduce). The 2 m route has been run only in simulation.

Figure 4. The 3 rounds on the robot, from its own record



Top: what each run did over time (hatched pink = steering in place, blue = driving, grey = still). The old method first steers ≈ 9.4 s in place, then drives; Detour Steer drives at once and steers less later. **Bottom:** the paths

the robot measured (odometry), all six from one start, seen from above; dashed = the plan. (The robot steers counter-clockwise, so here the goal is to the right.)

Appendix A. Code and Git repository

All code, the simulators, the firmware and this report are in one public Git repository:

`https://github.com/Mannaja78520/mor_luam` (branch `main`)

Video of the robot working:

`https://drive.google.com/drive/folders/11qOM6UoeH2c4nEuM0a0JO6eXTnr0eDud`

The clip shows the robot working in synchronize mode. There is no clip of the Detour Steer vs old method (Direct) comparison; that comparison comes from the robot's own record (Figure 4 and the table of results on the robot).

Where the decision algorithm lives

- `algorithm_sim/detour_steer.py` – reference planner, theorem checks T1–T4, benchmark, figures.
- `algorithm_sim/sim_detour_steer.py`, `sim_detour_only.py` – desktop simulators.
- `firmware/src/algorithm/DetourSteer.{h,cpp}` – the same planner in C++ on the ESP32.
- `firmware/src/nav/WaypointRunner.cpp` – runs the plan leg by leg on the robot; `FinalApproach.h` and `SteerApproach.h` – Algorithms 1.6 and 1.7.

Get the code and run the simulator (no Docker needed)

```
git clone https://github.com/Mannaja78520/mor_luam.git
cd mor_luam/algorithm_sim
pip install matplotlib
python detour_steer.py --no-bench      # theorem checks T1-T4 + figures
python sim_detour_steer.py             # old method vs Detour Steer
python -m unittest test_current_robot
```

Build and flash the robot firmware (Docker recommended)

```
docker\mor_luam.bat fw-build              # Windows, Docker Desktop running
docker\mor_luam.bat fw-flash -Port COM9   # first time, over USB
docker\mor_luam.bat fw-ota -Host <IP>     # later, over Wi-Fi
./docker/mor_luam.sh fw-build             # Ubuntu, same commands
```

Step-by-step install guide: `docs/INSTALL_TH.md` in the repository.

A.1 Planner in Python (`algorithm_sim/detour_steer.py`)

```python
def optimal_detour_angle(phi: float, d: float, robot: RobotConfig) -> Optional[float]:
    """beta* in (pi, phi), or None when no detour can beat steering straight.

    phi: counter-clockwise angle from the wheel to the goal (0 .. 2*pi)
    d  : distance to the goal (m)
    """
    if phi <= math.pi:                       # T1: straight at the goal is optimal
        return None
    w = math.radians(robot.steer_speed_dps)
    k = (d * w / robot.drive_speed_mps) * abs(math.sin(phi))
    if k >= 2.0:                             # T3: time falls all the way to phi
        return None
    beta = TAU - math.acos(k - 1.0)          # T3: the one root of dT/dbeta = 0
    return beta if beta < phi else None


def detour_legs(phi: float, d: float, beta: float) -> Tuple[float, float]:
    """Lengths of leg 1 (heading 0) and leg 2 (heading beta) that end at the goal."""
    s = math.sin(beta)
    return d * math.sin(beta - phi) / s, d * math.sin(phi) / s


# ====================================================================
# Algorithm 1.3: plan one leg (direct or detour)
# ====================================================================


@dataclass(frozen=True)
class Plan:
    kind: str            # "direct" or "detour"
    a: float             # leg 1, driven first with NO steer (0 for direct)
    beta: float          # the one counter-clockwise steer
    b: float             # leg 2, to the goal
    time_s: float


def direct_time(phi: float, d: float, robot: RobotConfig) -> float:
    """The current method: steer phi counter-clockwise, then drive d."""
    w = math.radians(robot.steer_speed_dps)
    steer = phi / w + robot.settle_s if phi > 1e-12 else 0.0
    return steer + d / robot.drive_speed_mps


def detour_time(a: float, beta: float, b: float, robot: RobotConfig) -> float:
    w = math.radians(robot.steer_speed_dps)
    v = robot.drive_speed_mps
    return a / v + robot.stop_s + beta / w + robot.settle_s + b / v


def plan_leg(phi: float, d: float, robot: RobotConfig) -> Plan:
    """O(1). The faster of 'steer phi, drive d' and the best detour."""
    direct = Plan("direct", 0.0, phi, d, direct_time(phi, d, robot))
    beta = optimal_detour_angle(phi, d, robot)
    if beta is None:
        return direct
    a, b = detour_legs(phi, d, beta)
    t = detour_time(a, beta, b, robot)
    # The detour adds one stop (drive -> steer), so it must really be faster.
    return Plan("detour", a, beta, b, t) if t < direct.time_s else direct
```

A.2 Planner on the robot (`firmware/src/algorithm/DetourSteer.h`)

```cpp
#pragma once
// Detour Steer - the algorithm of the 271401 homework
// (E:\271401\271401_pee_aut_homework\algorithm\DETOUR_STEERING\algorithm.pdf,
//  reference implementation: aut_HW_04_detour_steering\detour_steer.py).
//
// A wheel that steers one way only pays almost a full turn for a goal just
// "behind" its rotation. Two straight legs whose headings differ by more than
// 180 deg reach any direction, so instead of steering phi the robot may:
//   1. drive a metres along its current heading (no steer),
//   2. steer only beta* (180 < beta* < phi),
//   3. drive b metres to the goal.
//
```

```
//   k     = (d * omega / v) * |sin phi|
//   beta* = 360 - acos(k - 1)            only if phi > 180, k < 2, beta* < phi
//   a     = d * sin(beta* - phi) / sin(beta*)
//   b     = d * sin(phi) / sin(beta*)
//
// and the detour is taken only when it is really faster than steering
// straight (it costs one extra stop t_b). Theorems T1-T4 in the PDF: optimal,
// at most two legs, the closed form is the minimum, and planning again after
// leg 1 gives "steer beta*, drive b" - so the caller simply plans again after
// every move, and a wheel that stops PAST its angle is recovered the same way.
#include "algorithm/LegPlanner.h"

class DetourSteer : public LegPlanner {
public:
    const char* name() const override { return "detour"; }
    LegPlan plan(float phiDeg, float d, const RobotParams& p) const override;

    // Algorithm 1.2: beta* in degrees, or a negative value when no detour can
    // beat steering straight. Exposed for tests.
    static float optimalDetourAngleDeg(float phiDeg, float d, const RobotParams& p, float* kOut = nullptr);
};
```

A.3 Planner on the robot (`firmware/src/algorithm/DetourSteer.cpp`)

```
#include "algorithm/DetourSteer.h"
#include <math.h>
#include "algorithm/DirectPlanner.h"

namespace {
const float PI_F = 3.14159265358979f;
const float TAU_F = 2.0f * PI_F;
inline float rad(float deg) { return deg * PI_F / 180.0f; }
inline float deg(float r) { return r * 180.0f / PI_F; }
}  // namespace

float DetourSteer::optimalDetourAngleDeg(float phiDeg, float d, const RobotParams& p, float* kOut) {
    const float phi = rad(phiDeg);
    if (kOut) *kOut = 0.0f;
    if (phi <= PI_F) return -1.0f;                          // T1: straight at the goal is optimal
    const float w = rad(p.steerDps);
    const float k = (d * w / p.driveMps) * fabsf(sinf(phi));
    if (kOut) *kOut = k;
    if (k >= 2.0f) return -1.0f;                            // T3: time keeps falling up to beta = phi
    const float beta = TAU_F - acosf(k - 1.0f);             // T3: the one root of dT/dbeta = 0
    return beta < phi ? deg(beta) : -1.0f;
}

// Algorithm 1.3 (PlanLeg): the faster of "steer phi, drive d" and the best detour.
LegPlan DetourSteer::plan(float phiDeg, float d, const RobotParams& p) const {
    LegPlan direct = DirectPlanner().plan(phiDeg, d, p);
    float k = 0.0f;
    const float betaDeg = optimalDetourAngleDeg(phiDeg, d, p, &k);
    direct.k = k;
    if (betaDeg < 0.0f) return direct;

    const float phi = rad(phiDeg), beta = rad(betaDeg);
    const float s = sinf(beta);
    const float a = d * sinf(beta - phi) / s;               // leg 1: heading unchanged
    const float b = d * sinf(phi) / s;                      // leg 2: heading beta*, ends at the goal
    const float t = a / p.driveMps + p.stopS + betaDeg / p.steerDps + p.settleS + b / p.driveMps;
    if (!(t < direct.timeS)) return direct;                 // the extra stop must pay for itself

    LegPlan l;
    l.kind = LegPlan::Detour;
    l.a = a;
    l.betaDeg = betaDeg;
    l.b = b;
    l.timeS = t;
    l.k = k;
    return l;
}
```